اصول طراحی پایگاه داده
نام و نام خانوادگی: علیرضا کریمی
۸۱۰۱۰۱۴۹۲

تمرین شماره ۹

سوال ۱

الف)

این ۴ روش را داریم:

- ۱. $Single - Index\ Access\ Path$

تک شاخصی را انتخاب می‌کنیم که کمترین هزینه IO را دارد. و با استفاده از آن، $tuple$ ها را به حافظه اصلی آورده و محاسبات را انجام می‌دهیم.

- ۲. $Multiple - Index\ Access\ Path$

اگر بیش از یک شاخص داریم که با شرایط ما $match$ می‌شود، و $Alt.2$ یا $Alt.3$ هستن، می‌آییم rid ها را اشتراک می‌گیریم و آنهایی که همه شرایط را دارند، به حافظه اصلی میاوریم.

- ۳. $Sorted\ Index\ Access\ Path$

اگر شاخصی داریم که $result$ ها بر اساس یک $attribute$ مد نظر ما $sort$ شده هستن، از آن شاخص استفاده می‌کنیم.

- ۴. $Index - only\ Access\ Path$

اگر شاخصی داریم که با تنها با کمک $data\ entry$ ها می‌تونیم به آن $query$ پاسخ دهیم، از آن شاخص استفاده می‌کنیم.

ب)

در عمل چند حالت مختلف وجود دارد که در آن‌ها استفاده از $full\ table\ scan$ به‌جای $index$ بهینه‌تر است:

۱. فقدان یا نامناسب‌بودن شاخص

اگر هیچ شاخصی ($index$) روی جدول نداشته باشیم ناچار به اسکن کامل جدول هستیم.

حتی اگر شاخص وجود داشته باشد، ممکن است جدول مرتباً در حال تغییر باشد یا آمار ($statistics$) و $index$ های موجود نمایانگر الگوی واقعی داده‌ها نباشند؛ در این صورت هم $full\ table\ scan$ معمولاً بهتر است.

۲. نرخ بازگشت ($fetch$) بالای داده‌ها

وقتی پیش‌بینی می‌کنیم که پرس‌وجو بیش از نیمی از ردیف‌های جدول را برمی‌گرداند، با هر $I/O\ call$می‌توانیم بلوک‌های پیاپی زیادی را خوانده و کل عملیات اسکن را در تعداد محدودی I/O خلاصه کنیم. در این وضعیت، استفاده از $index$ (به‌ویژه $unclustered\ index$) به‌دلیل نیاز به I/Oهای پراکنده (به ازای هر $tuple$ و نه به ازای هر $page$) پرهزینه‌تر خواهد بود.

۳. شرط‌های پیچیده یا ترکیبی

اگر شرط‌های پرس‌وجو با اپراتورهای 'OR' یا بر روی ستون‌هایی نوشته شده باشد که روی آن‌ها $index$ تعریف نکرده‌ایم (یا $index$ مناسب ندارند)، نیاز به بررسی و اسکن کل داده‌ها داریم. مثلا فرض کنید که دو ستون داریم که بر روی یکی از آنها شاخص داریم اما بر روی دیگری شاخص $selective$ نداریم و یا کلا شاخص نداریم، به گونه‌ای که هزینه پاسخ به آن، مجموع هزینه‌های آن‌ها خواهد شد و ممکن است از اسکن کلی هم بیشتر شود. در این موارد $full\ table\ scan$ ساده‌تر و سریع‌تر است.

در مجموع، وقتی یا به شاخص دسترسی نداریم، یا $selectivity$ (نسبت داده‌های برگردانده شده به کل جدول) بسیار بالا است، یا ساختار شرط‌ها مانع استفاده مؤثر از $index$ می‌شود، اسکن کامل جدول ($full\ table\ scan$) کم‌هزینه‌تر و عملی‌تر خواهد بود.

ج)

$most\ selective\ access\ path$ شاخص یا اسکنی است که تخمین می‌زنیم کمترین تعداد $block$ ها را می‌خواند و کمترین هزینه I/O را دارد. درواقع $most\ selective\ access\ path$، مسیر دسترسی به پرس‌وجویی است که کمترین تعداد پیج را در حین ارزیابی پرس‌وجو $retrieve$ می‌کند. در مورد این مفهوم می‌توانیم به $reduction\ factor$ هم اشاره کنیم. آن شاخصی را انتخاب می‌کنیم که مقدار RF کمتری از بقیه دارد و بیشتر داده‌ها را غربال و $select$ می‌کند. مثلا بین دو شاخصی که یکی ۱۰ درصد داده و دیگری ۱ درصد داده‌ها را $select$ می‌کنند، داده‌ای را که تنها ۱ درصد داده‌ها را $select$ می‌کند، انتخاب می‌کنیم تا داده‌های بیشتری حذف شوند و ادامه محاسبات بر روی داده‌های کمتری انجام شود.

در پاسخ به این سوالات، از اسلایدهای ۱۴ و ۱۵ درس و همچنین تمارین سال‌های گذشته کمک گرفته‌ام.

سوال ۲

ابتدا تعداد صفحات هر کدام از $relation$ ها را محاسبه می‌کنیم:

- برای $Write$ داریم: $\lceil \frac{2000}{10} \rceil = 200\ ps$
- برای $BookContribution$ داریم: $\lceil \frac{10000}{10} \rceil = 1000\ ps$

• برای $Book$ داریم: $\lceil \frac{5000}{10} \rceil = 500\ ps$

ما در این $query$ باید $relation$ های $Write$ با ۲۰۰ صفحه و $BookContribution$ با ۱۰۰۰ صفحه را با یکدیگر $join$ کنیم. ۱۰۲ صفحه بافر داریم.

الف)

برای $POSNLJ$ داریم:

$$cost = \#Pages\ of\ Outer + \#Pages\ of\ Outer \times \#Pages\ of\ Inner$$

پس برای اینکه هزینه کمتری داشته باشیم، باید $relation$ی را برای $outer$ انتخاب کنیم که تعداد صفحات کمتری داشته باشد؛ پس داریم:

$$cost = N + N \times M = 200 + 200 \times 1000 = 200,200\ IO$$

ب)

برای $BNLJ$ دو حالت زیر را محاسبه می‌کنیم:

$$cost = \begin{cases} M + \lceil \frac{M}{B-2} \rceil \times N = 1000 + \lceil \frac{1000}{100} \rceil \times 200 = 1000 + 10 \times 200 = 3000\ IO \\ N + \lceil \frac{N}{B-2} \rceil \times M = 200 + \lceil \frac{200}{100} \rceil \times 1000 = 200 + 2 \times 1000 = 2200\ IO \end{cases}$$

پس در این حالت نیز کمترین هزینه برای زمانی است که رابطه کوچک‌تر در $outer$ و رابطه بزرگ‌تر $inner$ باشد، که برابر ۲۲۰۰ تا دسترسی به حافظه است.

ج)

برای استفاده از $hash\ join$ ساده، باید رابطه $B > \sqrt{min(M,N)}$ برقرار باشد. از آنجا که $50 > \sqrt{200}$، پس می‌توان از این روش استفاده کرد و داریم:

$$cost = 3 \times (M + N) = 3 \times (1000 + 200) = 3600\ IO$$

د)

برای استفاده از SMJ به صورت $refine$ شده، باید داشته باشیم $B > \sqrt{max(M,N)}$ و چون داریم $50 > \sqrt{1000}$ پس می‌توانیم از حالت $refine$ شده استفاده کنیم و داریم:

$$cost = 3 \times (M + N) = 3 \times (1000 + 200) = 3600\ IO$$

سوال ۳

خب، در ابتدا برای سادگی فرض می‌کنیم که توزیع همه رابطه‌ها برای تمامی ویژگی‌هاییشان، به صورت $uniform$ است. با توجه به اینکه تعداد صفحات بافر مشخص نشده است، فرض می‌کنیم که امکان استفاده از روش‌هایی که از بافر استفاده می‌کنند، وجود ندارد. بنابراین بسیاری از روش‌ها را نمی‌توانیم استفاده کنیم و تنها می‌توانیم از $INLJ$ استفاده کنیم. خب، در ابتدا برای سمت چپ‌ترین $relation$، باید آن رابطه‌ای را انتخاب کنیم که $selective$ ترین باشد. از طرفی باید درنظر داشته باشیم که برای مرحله بعد که باید با یک رابطه دیگر $join$ شود، این دو رابطه نمی‌توانند $Project$ و $Employee$ باشند، چون $attribute$ مشترک ندارند. پس در مرحله بعد حتما باید یکی از این دو با $Assignment$ ، $join$ شود. از طرفی از آنجایی که شاخص $Assignment$ روی eid آن است که مربوط به رابطه $Employee$ می شود، باید در سمت چپ‌ترین همان $Employee$ قرار بگیرد. (اگر بخواهیم رابطه‌ای که در سمت چپ‌ترین قرار می‌گیرد، کمترین تعداد $tuple$ را داشته باشد، برای $Employee$ داریم: $\lceil \frac{4,000 \times 15}{3} \rceil = 20,000\ ts$ و برای $Project$ داریم: $\lceil \frac{6,000 \times 12}{4} \rceil = 18,000\ ts$. همانطور که می‌بینید، با اینکه $Project$ تعداد $tuple$ کمتری دارد، اما چون در این صورت نمی‌توانیم از شاخص روی $Assignment$ استفاده کنیم، انتخاب $Project$ به عنوان چپ‌ترین رابطه، مردود است.)

حال برای پیدا کردن $Employee$ با $age < 40$ اگر از شاخص استفاده کنیم، چون $uncluster$ است، هزینه تعداد IO بسیار بیشتر از اسکن کل صفحات خواهد شد، پس از همان اسکن کل صفحات استفاده می‌کنیم که برابرست با:

$$cost = 4,000\ IO$$

بعد $projection$ می‌زنیم تا $attribute$های اضافی را حذف کنیم.

حال همانطور که تعداد $tuple$ ها را محاسبه کردیم که ۲۰۰۰۰ تا شد، به ازای هر کدام، باید بر روی $Assignment$، $look-up$ بزنیم. از آنجا که eid در $Assignment$ کلید نیست، و نسبت اندازه $Assignment$ به $Employee$ برابرست با: $\frac{12,000 \times 8}{4,000 \times 15} = 1.6$، پس به ازای هر $Employee$ ، ۱.۶ تا $Assignment$ وجود دارد و به ازای هر $look-up$ باید علاوه بر پیمودن ارتفاع درخت به اندازه ۳ برای پیدا کردن $data\ entry$، باید به اندازه ۱.۶ تا IO بزنیم چون این تعداد تاپل به ازای هر $Employee$ وجود دارد.

پس هزینه این کار برابرست با:

$$cost = 20,000 \times (3 + 1.6) = 92,000\ IO$$

و تعداد تاپل‌های خروجی برابرست با $20,000 \times 1.6 = 32,000\ ts$. حال بعد از عمل $pojection$، باید با $Project$، $join$ بزنیم با کمک $INLJ$. اما نکته اینجاست که به هیچ‌کدام از $attribute$ های $Project$ در خروجی نیاز نداریم و فقط باید شرط $budget > 5000$ را داشته باشند. پس همان $data\ entry$ شاخص برای ما کافی‌ست. از آنجا که pid در این رابطه کلید است، پس به ازای هر $look - up$، یک rid پیدا می‌شود. پس هزینه آن برابرست با:

$$cost = 32,000 \times (3 + 1) = 128,000\ IO$$

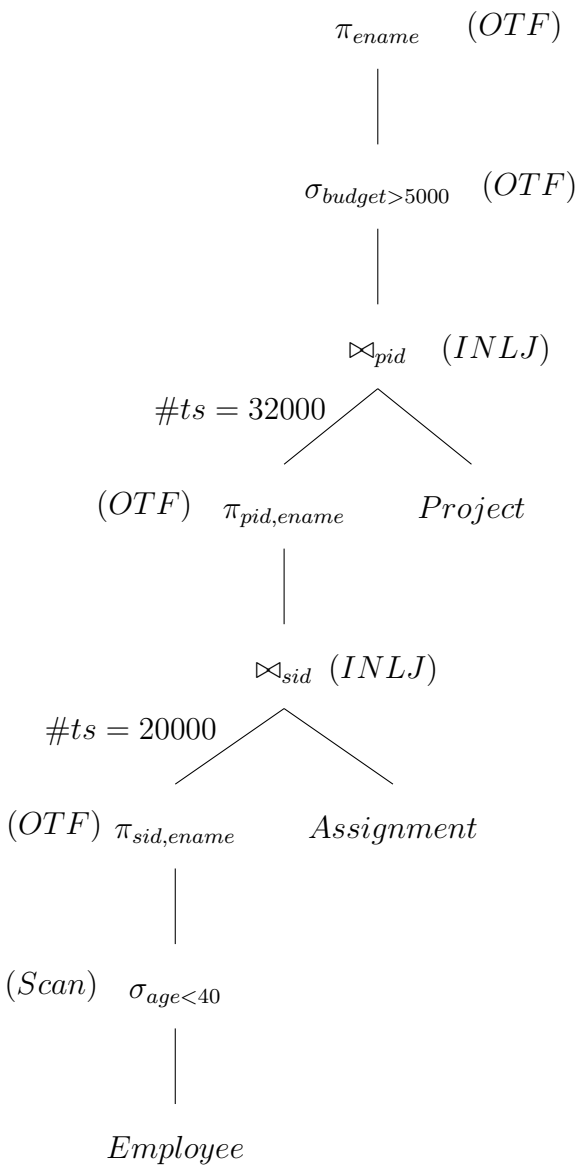
مثل قبل، $selection$ را OTF می‌زنیم. و همینطور $projection$ را نیز OTF می‌زنیم.
پس هزینه کل برابرست با:

$$Total\ cost = 4,000 + 92,000 + 128,000 = 224,000\ IO$$

اگر از روش‌های بافر دار استفاده می‌کردیم، هزینه به مراتب کمتر می‌شد. به عنوان مثال، در صورت داشتن بافر به میزان کافی، می‌توانستیم از $hash - join$ یا $refined\ sort - merge\ join$ نیز استفاده کنیم که هزینه $join$ آن‌ها از فرمول $3 \times (M + N)$ بدست میاید و به مراتب هزینه کمتری خواهد بود.
$plan$ بیان شده را می‌توانید در شکل ۱ مشاهده کنید.

سوال ۴

خب در ابتدا تعداد صفحات هر کدام از رابطه‌ها را حساب می‌کنیم و داریم که رابطه $User$، ۱۰۰ صفحه، رابطه $Booking$ ۵۰۰ صفحه و رابطه $Vehicle$ ۴۰۰ صفحه دارد.
از آنجایی که باید در انتها طبق $model$ دسته بندی کنیم، بهتر است در ابتدا طبق $model$، یک SMJ بزنیم تا طبق $model$ به صورت مرتب شده داشته باشیم و در انتها برای دسته بندی کردن نیاز به مرتب‌ساز دوباره نیست.
خب در ابتدا رابطه $Vehicle$ را می‌خوانیم با هزینه ۴۰۰ تا IO و بعد $selection$ می‌زنیم که باعث می‌شود تعداد صفحات، $\frac{1}{10}$ شود یعنی ۴۰ صفحه، بعد که $projection$ بزنیم، تعداد صفحات $\frac{1}{4}$ می‌شود که می‌شود ۱۰ صفحه. می‌توان این ۱۰ صفحه را به کمک ۲۵ صفحه بافر در همان $pass0$ مرتب کرد و به IO اضافی نیاز ندارد.
رابطه $Booking$ هم ۵۰۰ تا صفحه دارد که در $1 + \lceil log_{24}\lceil \frac{500}{50} \rceil \rceil (internal) = 1 + \lceil log_{24}\lceil \frac{500}{25} \rceil \rceil (external) =$ $2\ passes$ مرتب می‌شود پس درواقع می‌توان از $refined\ SMJ$ استفاده کرد، چون $25 > \sqrt{max(500, 400)}$، پس هزینه آن برابرست با:



شکل ۱: نمایش $plan$ بیان شده در سوال ۳

$$cost = 400 + 500 + 2 \times (10 + 500) = 1,920\ IO$$

حال، تعداد $tuple$های $result$ را محاسبه می‌کنیم. نسبت $Booking$ به $Vehicle$ برابرست با $\frac{200,000}{12,000} = \frac{50}{3}$. پس به ازای هر $tuple$ در $Vehicle$، ما تا $\frac{50}{3}$ $tuple$ در $Booking$ داریم. پس تعداد $tuple$های $result$ برابرست با:

$$\#tuples\ in\ result = 12,000 \times \frac{1}{10} \times \frac{50}{3} = 20,000\ ts$$

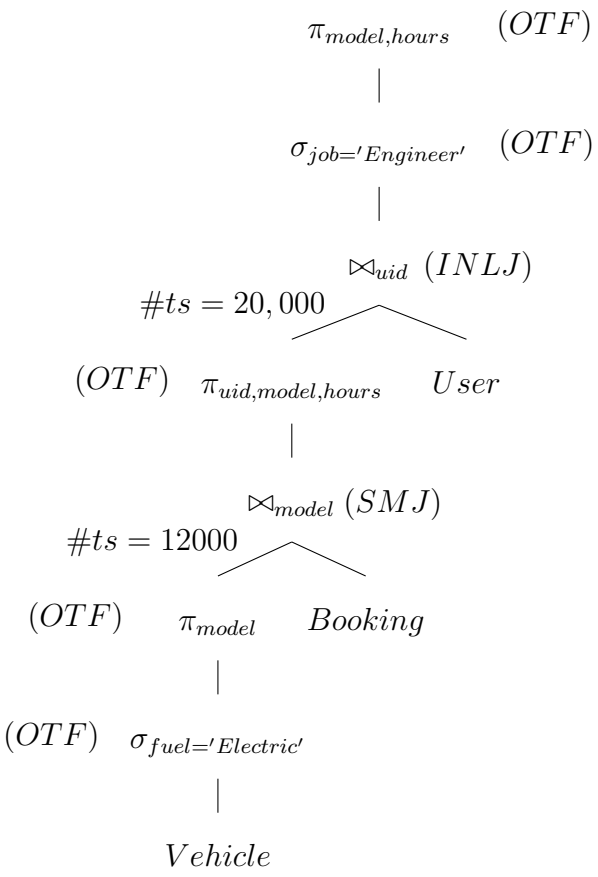
برای دور ریختن ویژگی‌های $useless$، یک $projection$ نیز به صورت OTF می‌زنیم. حال باید با رابطه $User$، $INLJ$ بزنیم. از آنجایی که uid در $User$ کلید است، پس از هر کدام از آنها در $User$ یک $tuple$ وجود دارد. به این معنی که به ازای هر کدام از آن ۲۰،۰۰۰ تا $tuple$، یک $tuple$ با $look-up$ پیدا می شود. ارتفاع درخت ۳ است. پس هزینه این کار برابرست با:

$$cost = 20,000 \times (3+1) = 80,000\ IO$$

بعد از آن، $selection$ و $projection$ های گفته شده را به صورت OTF انجام می‌دهیم.
بنابراین هزینه کل برابرست با:

$$Total\ cost = 1,920 + 80,000 = 81,920\ IO$$

در شکل ۲ می‌توانید $plan$ مربوط به این سوال را مشاهده کنید.



شکل ۲: نمایش $plan$ بیان شده در سوال ۴

سوال ۵

می‌دانیم هزینه خواندن ابتدایی $Product$ برابر خواهد بود با $4,000\ IO$. در ابتدا باید روی رابطه $Product$، $selection$ بزنیم. طبق چیزی که در پلن نشان داده شده، ۱۹۹ مقدار ابتدایی $price$ را داریم، اما برای راحتی کار، فرض می‌کنیم که ۲۰۰ مقدار $price$ مختلف داریم که معادل با $\frac{1}{5}$ تعداد کل $price$هاست. تعداد تاپل‌ها بعد از $selection$ برابر خواهد بود با:

$$4,000\ ps \times 10\ tps \times \frac{1}{5}\ (RF) = 8,000\ ts$$

از آنجایی که نسبت $Order$ به $Product$، برابرست با $\frac{8000\times10}{4000\times10} = 2$، یعنی به ازای هر تاپ در $Product$، ۲ تاپل در $Order$ پیدا می‌شود. پس هزینه $INLJ$ کردن به کمک شاخص $unclustered - hash$، برابرست با:

$$cost = 8,000 \times (2+2) = 32,000\ IO$$

تعداد تاپل‌های خروجی نیز برابر خواهد بود با $8,000 \times 2 = 16,000\ ts$

حال دوباره باید به ازای تک‌تک این تاپل‌ها، یک $look - up$ بر روی شاخص $unclustered - tree$ روی uid بزنیم و از آنجایی که uid کلید است، هر $look - up$ به ما یک تاپل خواهد داد. پس هزینه $INLJ$ در این مرحله نیز برابر خواهد بود با:

$$16,000 \times (3+1) = 64,000\ IO$$

بعد، $selection$ و $projection$ را به صورت OTF انجام می‌دهیم که هزینه‌ای نخواهند داشت.

پس در کل هزینه این پلن برابرست با:

$$Total\ cost = 4,000 + 32,000 + 64,000 = 100,000\ IO$$